

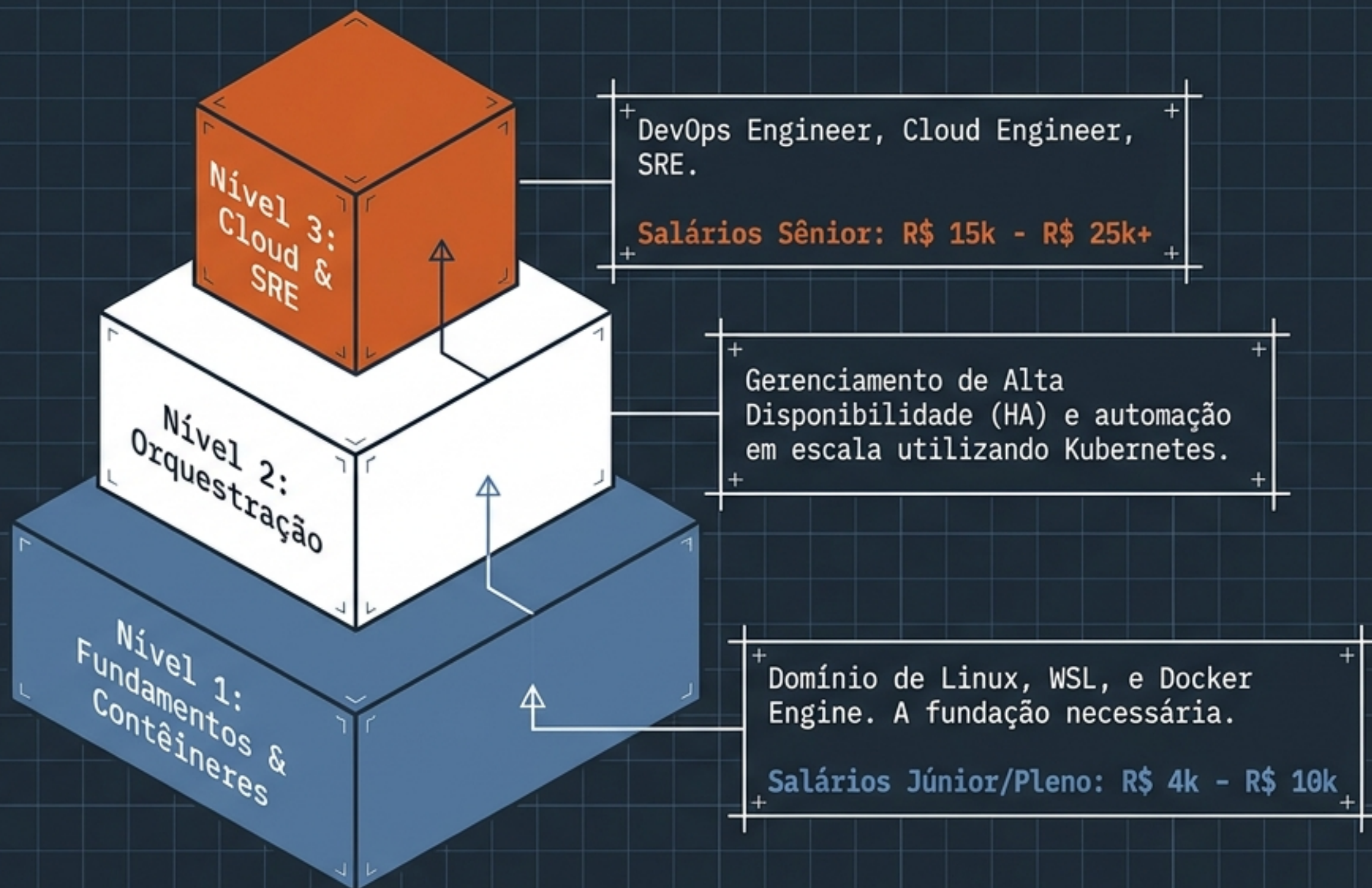
MÓDULO V | AULA 9



Anatomia de um Titã: Fundamentos de Kubernetes

Orquestração, Alta Disponibilidade e o Padrão
Ouro da Infraestrutura Moderna.

O EPICENTRO DA INFRAESTRUTURA MODERNA E DA CARREIRA DEVOPS



O Kubernetes não é apenas uma ferramenta; é o sistema operacional da nuvem moderna. Ele garante que as aplicações nunca caiam, escalem sob demanda e sobrevivam a falhas de hardware de forma autônoma.

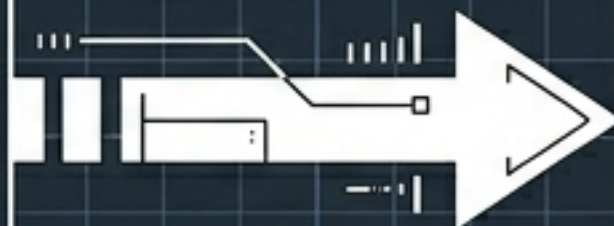
ARQUITETURA DO CLUSTER: O CÉREBRO E O MÚSCULO

CONTROL PLANE (O CÉREBRO)



Função: Mantém o estado global, toma decisões de alocação e responde a eventos do sistema.

Regra de Ouro: Nunca executa as aplicações dos usuários. Apenas gerencia.



WORKER NODES (O MÚSCULO)



Função: Os cavalos de carga. Máquinas virtuais ou físicas que efetivamente rodam os contêineres da aplicação.

Escalabilidade: Podem existir dezenas, centenas ou milhares em um único cluster.

Essa separação garante que, mesmo que todo o poder de processamento caia, a inteligência do sistema permaneça intacta para coordenar a recuperação.

ANATOMIA DO CONTROL PLANE (MASTER)

1. A Entrada

`kubectl apply`

O desenvolvedor envia um manifesto YAML.

2. API Server

O único componente que recebe chamadas diretas. Ele autentica, valida e atua como o maestro exclusivo do cluster.

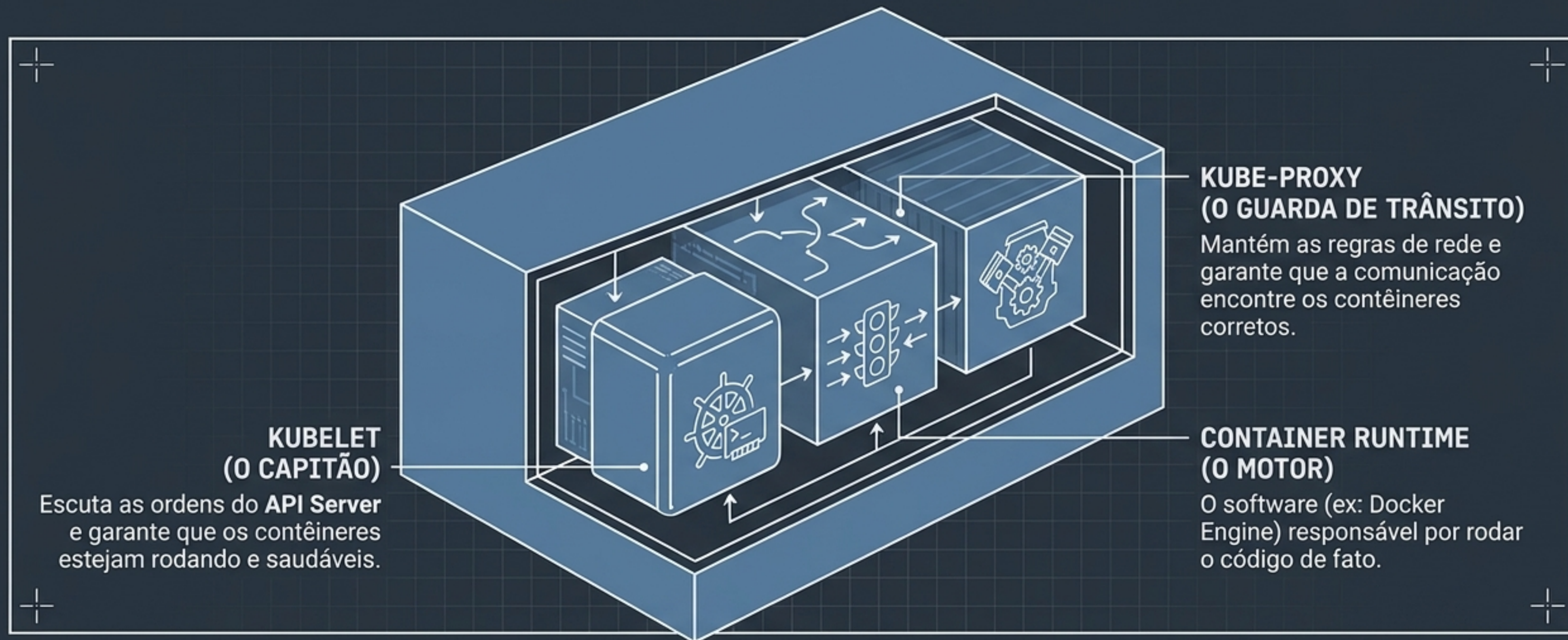
3. O Cofre Forte (etcd)

O banco de dados chave-valor ultrarrápido que guarda o "Estado Desejado" de todo o cluster. A única fonte da verdade.

4. O Estrategista (Scheduler)

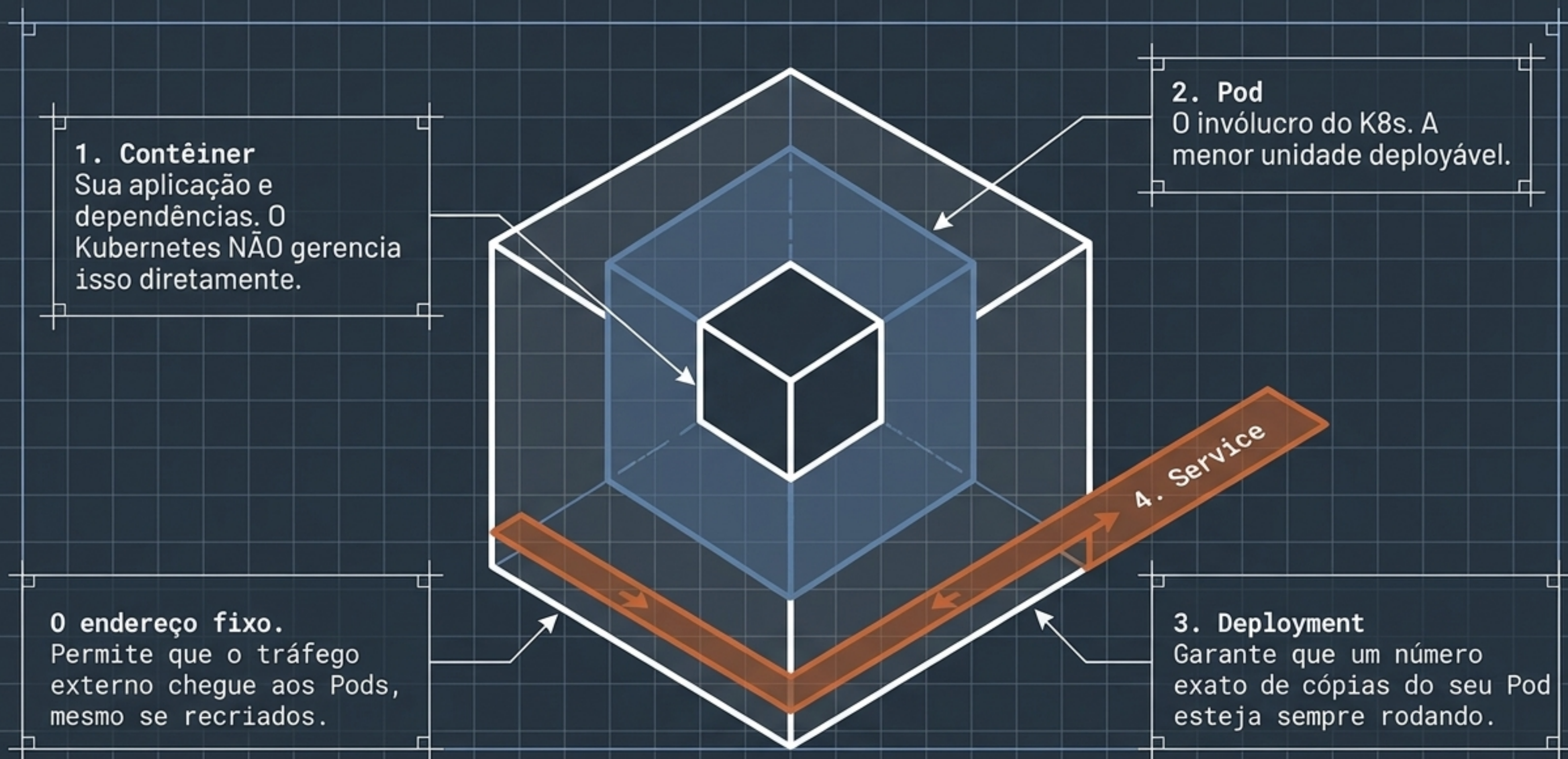
Observa os novos pedidos de trabalho que ainda não têm uma máquina atribuída e decide em qual Worker Node eles devem rodar.

ANATOMIA DO WORKER NODE



✦ O Worker Node é um ambiente de execução obediente. Se o Kubelet perde contato com o Control Plane, ele continua rodando o que já tem, mas não pode aceitar novas ordens. ✦

WORKLOADS: O MODELO DE MATRYOSHKA



O Pod: A Unidade Atômica e Efêmera



Isolamento Mínimo

É a menor unidade de computação deployável que você pode criar e gerenciar no Kubernetes.

Mortalidade (Efemeridade)

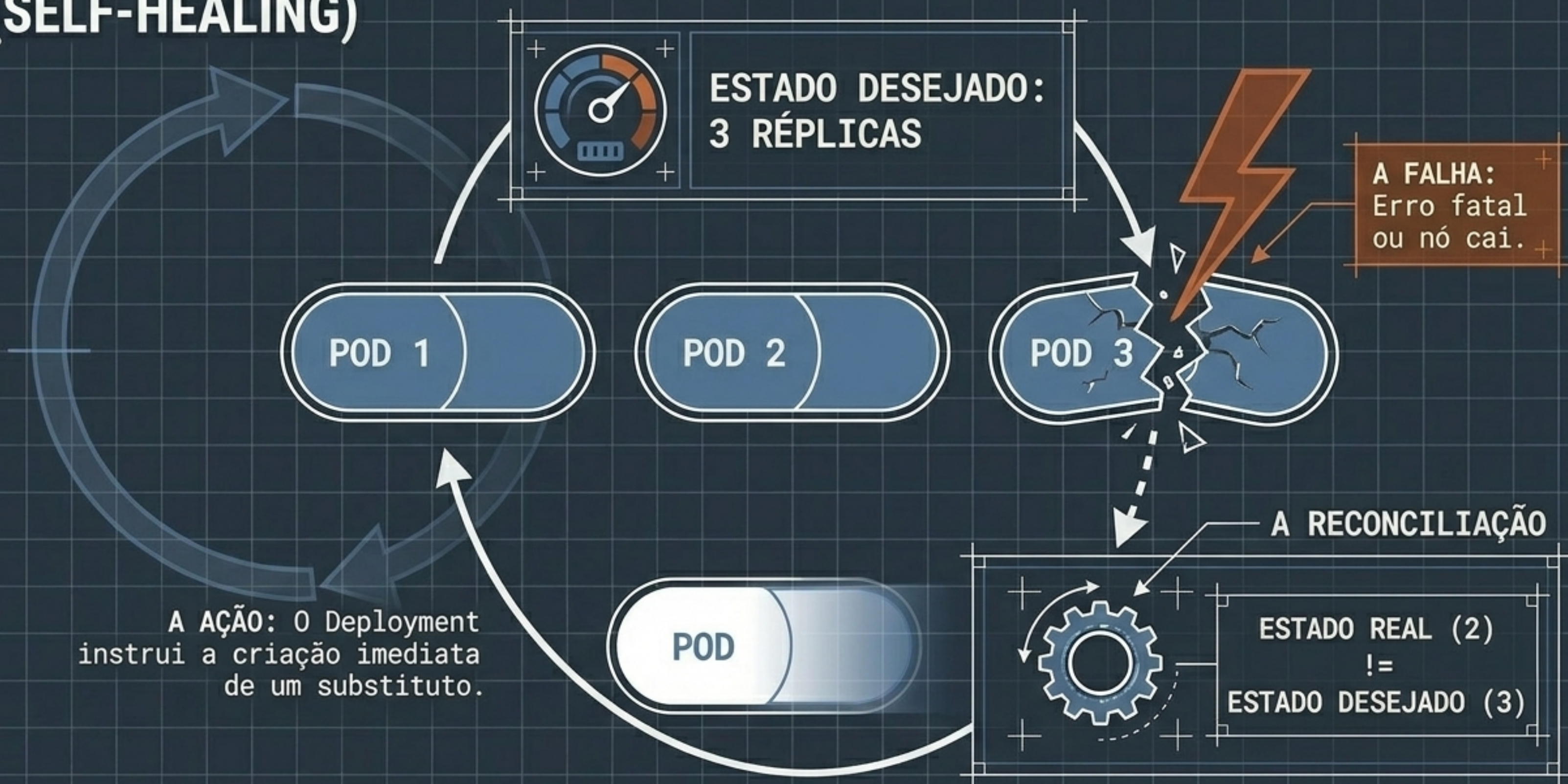
Pods não são servidores de estimação (Pets), são gado (Cattle). Eles nascem, morrem, e são substituídos. Eles não são consertados, são descartados.

IPs Descartáveis

Quando um Pod morre e é substituído, o novo Pod recebe um IP completamente diferente.

REGRA DE DESIGN: Nunca dependa diretamente do endereço IP de um Pod para comunicação em produção.

DEPLOYMENTS & O LOOP DE AUTO-CURA (SELF-HEALING)



Services: A Ponte Estável sobre a Névoa



O Problema:



Pods morrem e mudam de IP o tempo todo. Como os usuários os encontram?

A Solução:



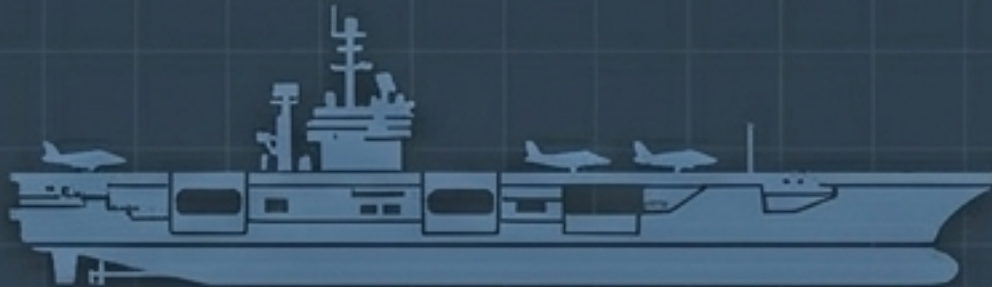
O Service atua como um balanceador de carga interno. Ele recebe um IP fixo permanente.

A Mecânica:



Rastreia os Pods ativos via 'Labels'. Redireciona o tráfego apenas para Pods saudáveis, mascarando o caos.

Matriz de Diagnóstico: Swarm vs. Kubernetes

	 Docker Swarm	 Kubernetes
Complexidade de Setup	Baixa ('Init' rápido, curva de aprendizado suave).	Alta (Requer infraestrutura densa, curva de aprendizado íngreme).
Arquitetura	Managers/Workers integrados diretamente ao Docker Engine.	Control Plane e Workers divididos com componentes autônomos pesados.
Ecossistema	Limitado, utiliza apenas as ferramentas embutidas.	Massivo, é o padrão da indústria com integrações infinitas (CNCF).
Casos de Uso	Startups, ambientes de teste, projetos ágeis menores.	Aplicações corporativas complexas, microsserviços em produção na nuvem.
VEREDITO: Swarm é focado na velocidade e simplicidade; Kubernetes é construído para resiliência e operação contínua em hiperescala.		

O Desafio do Armazenamento: Volumes no K8s



O Paradigma: Se os Pods são efêmeros, os dados de bancos de dados não podem viver dentro deles.



A Solução: Fatias de armazenamento provisionadas por StorageClasses que existem independentemente do ciclo de vida de qualquer Pod.

Anchor/Mount

PersistentVolume (PV) / AWS EBS

No K8s, a computação (Pods) e os dados (Volumes) possuem ciclos de vida completamente separados e isolados.

DA TEORIA À PRÁTICA: O CONTROLE REMOTO

O Controle: kubectl

A interface de linha de comando oficial.
O seu "controle remoto" no WSL para falar com a API.

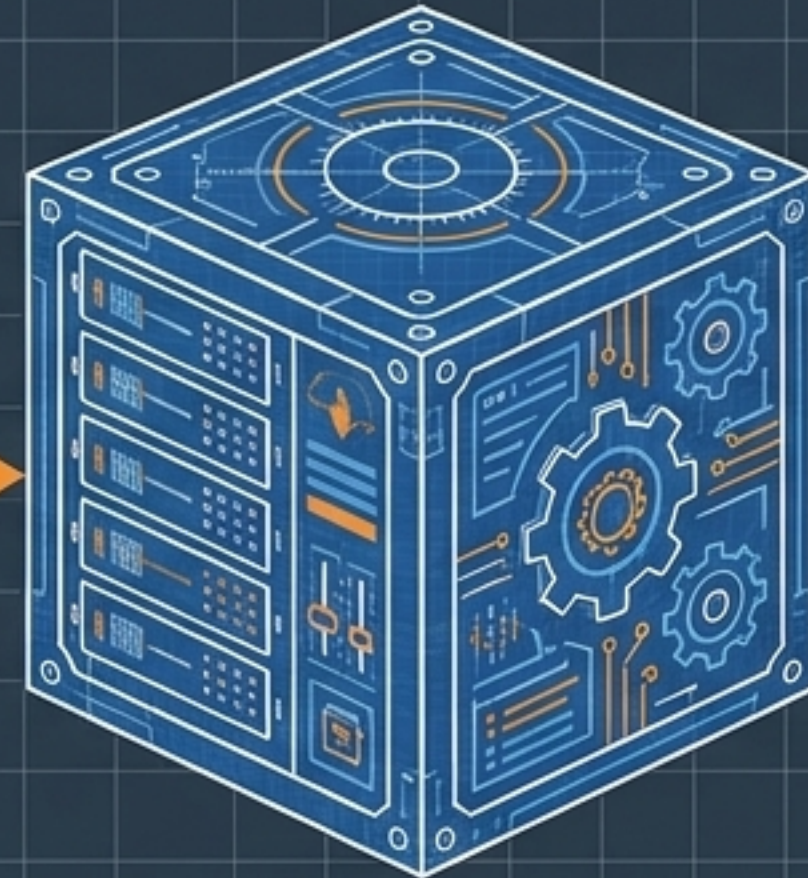
Comandos:

```
kubectl get pods, kubectl apply -f
```

Versões enxutas do Kubernetes projetadas para rodar no seu próprio WSL2/Docker. Permitem testar a arquitetura completa antes do deploy em nuvem.



Comunica via API REST

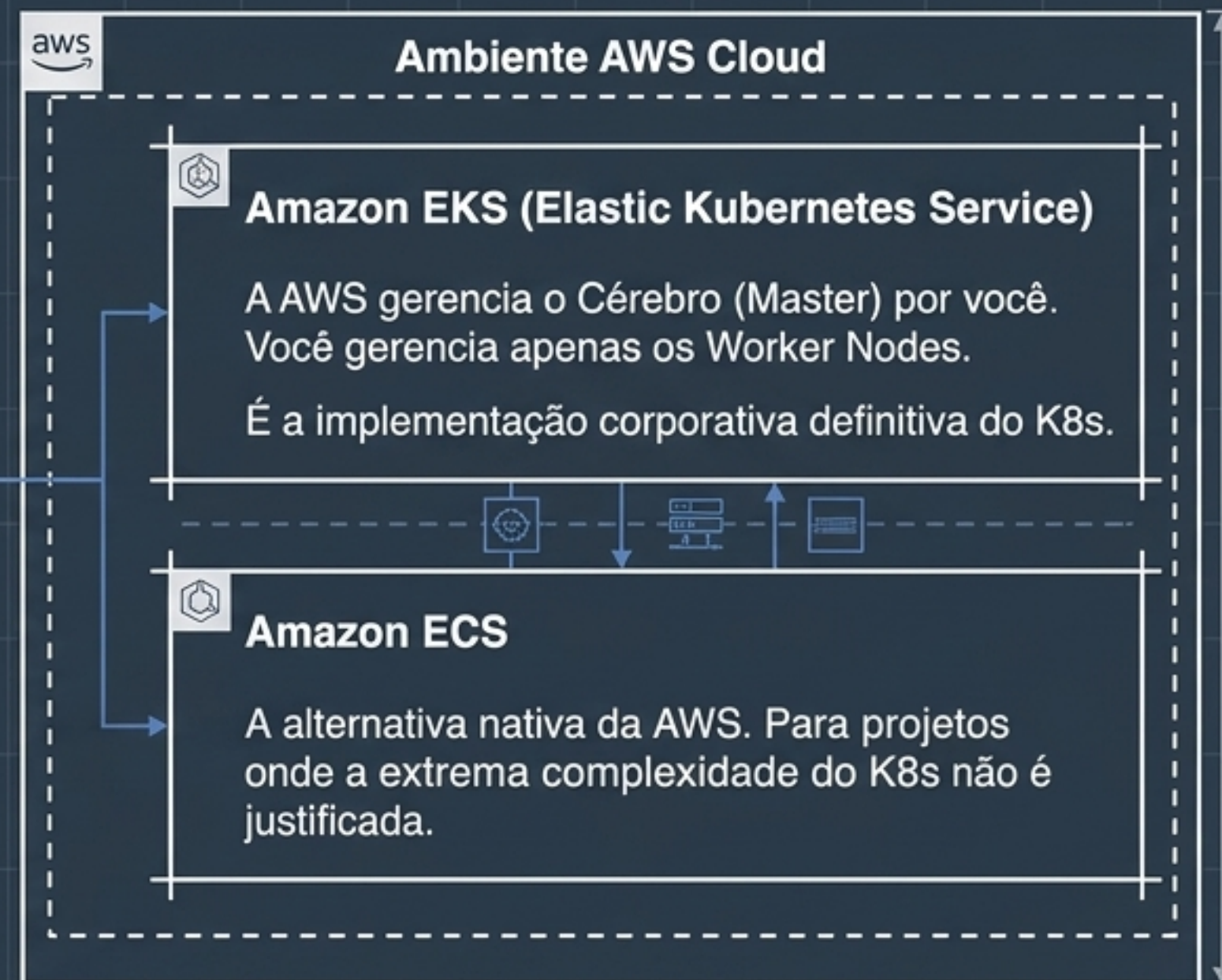
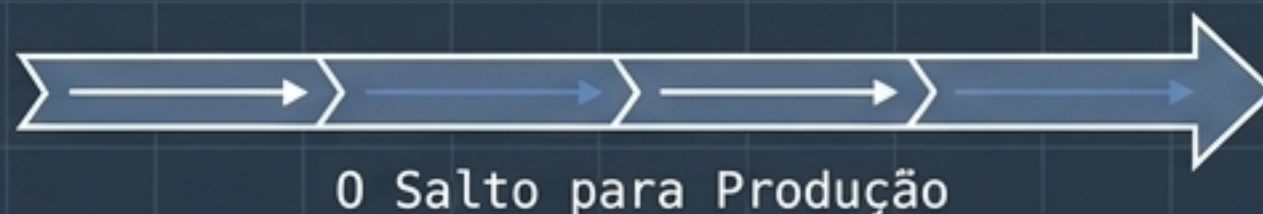


O Laboratório Local:
Minikube / K3s

O Horizonte em Nuvem: Kubernetes na AWS



WSL / Minikube



O domínio rigoroso de Docker e K8s local é a ponte absoluta para implementações maduras, escaláveis e seguras na nuvem AWS nos próximos módulos.

CHECKPOINT DA MISSÃO: PREPARAÇÃO PARA OS LABORATÓRIOS



Compreender a Arquitetura: Master vs. Worker Nodes claramente mapeados e entendidos.



Dominar os Workloads: Distinguir o ciclo de vida entre Pods (efêmeros) e Deployments (controladores).



Entender a Comunicação: Como os Services mantêm a acessibilidade sob caos.



Preparar o Arsenal: Garantir que o `kubectl` e o ambiente WSL estejam operacionais.

PRÓXIMA ETAPA: Provisionamento de Nuvem com AWS EC2 e VPC (Aula 10).